Computational Optimization and Applications (2026) 93:489–521
https://doi.org/10.1007/s10589-025-00739-z

Active set-based inexact proximal bundle algorithm for stochastic quadratic programming

Niloofar Fadavi[1] · **Harsha Gangammanavar**[1]

Received: 28 June 2024 / Accepted: 25 September 2025 / Published online: 17 October 2025
© The Author(s) 2025

Abstract
In this paper, we examine two-stage stochastic quadratic programming problems, where the objective functions of the first and second stages are quadratic, and the constraints are linear. The uncertainty is associated with the second-stage right-hand side and variable bounds. In large-scale settings, when the number of scenarios necessary to represent the underlying stochastic process is exuberantly large, standard decomposition-based methods that require the exact solutions are computationally prohibitive. To address this issue, we develop two inexact proximal bundle algorithms that rely on the efficient reuse of solution information. The first algorithm utilizes a collection of previously encountered second-stage dual solutions to construct inexact minorants for the expectation-valued objective function. On the other hand, a partition-based inexact proximal bundle algorithm utilizes the optimal active sets obtained in earlier iterations, along with a primal-dual active set method, to construct the inexact minorant. For both these variants, we establish their asymptotic convergence to optimal solutions. Using a carefully developed computer implementation, we demonstrate the practical behavior of these algorithms through numerical experiments conducted on power systems planning and operations problems. The results indicate that the partition-based algorithm consistently identifies solutions of comparable quality to those obtained from exact algorithms, while significantly reducing computational time.

Keywords Stochastic programming · Quadratic programming · Decomposition methods · Active-set methods · Inexact solution algorithms

✉ Harsha Gangammanavar
harsha@smu.edu

Niloofar Fadavi
nfadavi@smu.edu

[1] Operations Research and Engineering Management, Southern Methodist University, Caruth Hall, 3145 Dyer St, Dallas, TX 75205, USA

1 Introduction

In this paper, we study the 2-SQP problem that is stated as follows:

$$\begin{aligned} \min\ & f(x) := \frac{1}{2}x^\top Px + c^\top x + \mathbb{E}[h(x,\tilde{\omega})] \\ \text{subject to } & x \in \mathcal{X} = \{x : Ax = b,\ \ell_x \le x \le u_x\}. \end{aligned} \tag{1a}$$

Here, $h(\cdot)$ is the optimal value of the following second-stage quadratic programming (QP) problem:

$$\begin{aligned} h(x,\omega) := \min\ & \frac{1}{2}y^\top Qy + d^\top y \\ \text{subject to } & Dy = \xi(\omega) - C(\omega)x,\ \underline{y}(\omega) \le y \le \bar{y}(\omega),\ y \in \mathbb{R}^{n_2}. \end{aligned} \tag{1b}$$

In the two-stage setting, the first-stage decision $x \in \mathbb{R}^{n_1}$ is determined before the realization of uncertainty that is represented by the random vector $\tilde{\omega}$. The first-stage objective is to minimize the sum of a quadratic function of x and the expected second-stage value function. The expectation is computed with respect to the probability space $(\Omega, \mathcal{F}, \mathbb{P})$ associated with the random vector $\tilde{\omega}$. Here, Ω is the set of sample space, $\mathcal{F}$ is the sigma-algebra, and $\mathbb{P}$ is the probability measure defined on $\mathcal{F}$. In the second stage, a QP problem is solved for a given first-stage decision x and a realization ω of the random vector $\tilde{\omega}$. The second-stage value function is often referred to as the recourse function. Note that the first-stage decision x affects only the right-hand side of the second-stage problem. In general, any of the second-stage parameters can be affected by $\tilde{\omega}$; however, we restrict our attention to problems where only the right-hand side element (i.e., ξ and C) and variable bounds ($\underline{y}$ and $\bar{y}$) depend on $\tilde{\omega}$.

When the support of Ω is finite, a two-stage stochastic programming (2-SP) problem can be reformulated as a deterministic extensive scenario problem and solved using off-the-shelf solvers. However, when the number of scenarios is large, such an endeavor is computationally prohibitive. Most algorithmic advances in stochastic programming (SP), particularly from a computational perspective, are targeted at two-stage stochastic linear programming (2-SLP) problems, where the constraints and objective functions associated with the first-stage and second-stage optimization problems are linear. These advances exploit the decomposability of 2-SLP problems and can be classified into two primary groups: stage decomposition, exemplified by the L-shaped method [1], and scenario decomposition [2].

Similar stage and scenario decomposition approaches can be adopted for 2-SQP problems. For instance, [3] tackles a 2-SQP problem by dualizing the problem, approximating the dual by a sequence of lower-dimensional QP subproblems, and establishing that the optimal solutions sequence converges. In [4], a Newton-type method is proposed for 2-SQP. In [5], the authors extend the Newton-type method to exploit the decomposability of 2-SQP, particularly in computing the Newton directions. While these approaches address problems with finite support, [6] introduces an inexact Newton method and combines it with stochastic decomposition to address

2-SQP problems with continuous support. For the same class of problems, [7] establishes the sublinear convergence rate of stochastic decomposition.

To address the computational difficulty of cutting-plane methods, the inclusion of a quadratic regularization term is suggested. The resulting algorithm is often referred to as the proximal bundle algorithm. Introduced for deterministic convex programs by [8], the proximal bundle algorithm was adopted for SP problems in [9]. Inexact variants of the proximal bundle algorithm for deterministic convex programs have been studied in [10–12]. In SP, the inexact variants have been studied in [13], albeit only for 2-SLP. The regularized stochastic decomposition in [14] can also be viewed as an inexact proximal bundle algorithm, where the inexactness is due to its inherent stochastic design. While the original algorithm is for 2-SLP, [7] recently extended it to 2-SQP.

An alternative approach is the level bundle method, which was proposed by [15, 16] for deterministic convex programs. The inexact variants of the level bundle method have been studied, for example, in [17]. In [18], the authors study the inexact level methods that can be used to solve the problems to any desired level of accuracy. In the context of 2-SLP, level bundle methods have been applied, for instance, in [19, 20]. The techniques presented in this paper for approximating the second-stage quadratic recourse function can also be applied to solve 2-SQP using level bundle methods.

Our algorithm designs rely on efficiently solving QP problems that differ only along a few model parameters. Active-set methods, along with interior-point methods, are the primary methods for solving the QP problem with equality and inequality constraints. The active set methods reduce the problem size by identifying and excluding constraints that are not active in the current iteration. Various forms of active-set methods are presented in the literature based on the convexity properties of QP [21], including primal, dual, and primal-dual active-set (PDAS) methods. Primal and dual active-set methods preserve the feasibility of the primal and dual problems, respectively [22]. Regarding PDAS, [23] introduces a method where each iteration involves a partition of variable indices into an active set and an inactive set. The process results in a distinctive solution through a reduced KKT system. The partition is iteratively adjusted until the obtained solution satisfies both primal and dual feasibility conditions. This algorithm is presented for bound-constrained QP problems, and it is shown to cycle on more general convex QP problems. The algorithm outlined in [23] is the foundation for the methodologies presented in the following two works. In [24], an auxiliary index set is introduced alongside the active-set estimate. This auxiliary set is designated to contain the indices of variables for which bounds need to be explicitly enforced in a given iteration. This approach is globally convergent for strictly convex QP problems. In [25], convergence of the PDAS method is presented under certain assumptions on the structure of the QP problems even when the reduced KKT system is solved inexactly.

1.1 Contributions

In light of the current literature on decomposition methods for 2-SQP, this paper makes three main contributions. A good understanding of the sensitivity analysis and

solution properties of QP problems is essential for developing effective decomposition strategies. In this regard, we first analyze the solution properties of QP problems with specific right-hand sides. We establish that for small perturbations of the right-hand side within a polyhedron, the optimal partition of the variables into active and inactive sets does not change. Furthermore, we show that the partition that yields a feasible dual solution, which is necessary for constructing the approximation in bundle algorithms, remains the same for perturbations within a larger polyhedron. We adopt a PDAS method that, starting with an arbitrary partition, recovers a feasible dual solution in a computationally efficient manner.

Secondly, we present two variants of inexact proximal bundle algorithms that efficiently reuse the solution information from second-stage subproblems gathered in earlier iterations. The first method, referred to as a dual-based method, collects the dual solutions associated with second-stage problems in a set and generates a minorant based on this set. On the other hand, the second method, the partition-based method, involves collecting the optimal partitions obtained from previous iterations into a set and using the PDAS method over that set to generate the minorant. We note that our inexact bundle algorithms are specific manifestations of the general structure of inexact proximal bundle methods; see, for instance, the pattern identified in [12]. However, they differ in key aspects due to the randomized approach we adopt. Specifically, the criterion used to update the prox-center differs from that used in previous works on inexact proximal bundle methods. We recognize the similarities and highlight the differences after presenting our algorithms. Accounting for these differences, we establish asymptotic convergence of both the dual-based and partition-based inexact methods. Our analysis shows that the inexact procedures yield asymptotically exact minorants, thereby enabling asymptotic convergence of the overall methods without noise attenuation (see [12]). Finally, we also demonstrate that the inexact variants generate solutions of comparable quality and significantly reduce computational time when compared to the exact proximal bundle algorithm.

In addition to algorithm design and convergence analyses, a supplementary contribution of this work is the general-purpose implementation of the variants of proximal bundle algorithms in the C programming language. Our implementation leverages the problem features and dedicated data structures to manage memory assignments efficiently. This carefully designed implementation minimizes computational overhead by storing and reusing frequently accessed information, such as dual solutions and partitions, and allows us to reduce the overall computational workload.

1.2 Organization

We organize the paper as follows. In Sect. 2, we analyze the solution properties of QP problems and relate them to the notion of variable partitions. We also adopt a PDAS method to recover solutions in a computationally efficient manner. In Sect. 3, we introduce the inexact proximal bundle algorithms and present their asymptotic analyses. We present the results from the numerical experiments in Sect. 4. Finally, in Sect. 5, we discuss the conclusions drawn from our study and propose potential areas for future research. Proofs for most of the results are included in "Appendix A".

2 Solution properties of QP problems

Our aim is to develop decomposition-based solution algorithms for 2-SQP. To achieve that, we first analyze QP problems in this section. We then apply this analysis in Sect. 3 to present the solution methods for 2-SQP. For our discussion, we consider the following QP problem:

$$\begin{aligned} \min\ & g_P(y;x,\omega) := \frac{1}{2}y^\top Qy + d^\top y \\ \text{subject to}\ & Dy = \rho,\ \underline{y} \le y \le \bar{y}. \end{aligned} \tag{2}$$

The above problem is obtained from (1b) by substituting the right-hand side of the equality constraints with $\rho = \xi(\omega) - Cx$ and the lower and upper bounds with $\underline{y} = \underline{y}(\omega)$ and $\bar{y} = \bar{y}(\omega)$. We assume that Q is a positive semidefinite matrix, in which case the linear-quadratic objective function $g_P(y;x,\omega)$ is a convex function. We denote the sets of variable and constraint indices as $\mathrm{N} := \{1,\ldots,n_2\}$ and $\mathrm{M} := \{1,\ldots,m_2\}$, respectively. It is worth noting that the QP problem presented above resembles the second-stage problem in (1b). For ease of notation, we assume that C and $\underline{y}$ are independent of ω, although the subsequent outcomes remain applicable even if they are dependent.

An important property of (2) is that the feasible region of the dual problem is independent of the right-hand side parameters. The dual of the QP problem is given by:

$$\begin{aligned} \max\ & g_D(\pi;x,\omega) = -\frac{1}{2}u^\top Qu + \rho^\top\lambda + \underline{y}^\top\nu - \bar{y}^\top\mu \\ \text{subject to}\ & D^\top\lambda + \nu - \mu - Qu = d,\ \nu,\mu \ge 0, \end{aligned} \tag{3}$$

where $\pi := (u,\lambda,\nu,\mu)$ denotes the dual vector. The above dual is the type II form of QP dual given in [26]. According to [26], if y^* is the optimal solution of (2), there exists an optimal solution for (3) such that $u^* = y^*$. Therefore, we use $u = y$ in the remainder of this paper when referring to the dual problem. Let λ denote the dual variables (Lagrange multiplier) associated with the equality constraints, and μ and ν are dual variables associated with upper and lower bounds, respectively.

2.1 Solution recovery using decomposed KKT system

In addition to the aforementioned property, analyzing the effect of the QP problem's parameters on the first-order KKT optimality conditions reveals additional useful properties. For the QP in (2), at optimality, the primal variable y and the dual variables (λ,μ,ν) satisfy KKT conditions that are given as:

$$Qy + d + \mu - \nu + D^\top\lambda = 0, \tag{4a}$$

$$Dy = \rho,\ \underline{y} \le y \le \bar{y}, \tag{4b}$$

$$0 \leq \mu \perp (\bar{y} - y) \geq 0, \tag{4c}$$

$$0 \leq \nu \perp (y - \underline{y}) \geq 0. \tag{4d}$$

In equations (4c) and (4d), the symbol $\perp$ denotes a complementary relationship, indicating that the non-negative Lagrange multipliers μ and ν are complementary to the slackness conditions between the primal variable y and its upper bound $\bar{y}$ and between y and its lower bound $\underline{y}$.

A primal optimal solution y to (2) has elements either at their lower-bound ($y_i = \underline{y}_i$), upper-bound ($y_i = \bar{y}_i$), or in the interior of the interval ($\underline{y}_i < y_i < \bar{y}_i$). To distinguish these elements, we associate with a primal solution mutually exclusive and exhaustive subsets of indices denoted by L, U, and I that correspond to elements at lower-bound, upper-bound, and interior (inactive), respectively. That is

$$\mathrm{L} := \{i \mid y_i = \underline{y}_i, i \in \mathrm{N}\};\ \mathrm{U} := \{i \mid y_i = \bar{y}_i, i \in \mathrm{N}\};\ \mathrm{I} := \{i \mid \underline{y}_i < y_i < \bar{y}_i, i \in \mathrm{N}\}.$$

We refer to the set $\mathrm{A} = \mathrm{L} \cup \mathrm{U}$ as the *active* set and I as the *inactive* set. Since the mutually exclusive and exhaustive sets represent the partition of the index set N, we refer to the tuple $P = (\mathrm{L}, \mathrm{U}, \mathrm{I})$ as a *partition*. Specifically, we will refer to the partition associated with a solution that satisfies the KKT conditions in (4) as the *optimal partition*. In this paper , we use two index sets as subscripts to specify a sub-matrix of a matrix. For example, Q_{MB} indicates a sub-matrix of Q formed by rows and columns indexed by M and B, respectively.

The concept of partition is a useful tool in analyzing the KKT system. For a given partition P, the primal variable y, the dual variables (λ, ν, μ), and the problem parameters (Q, D, d) can be decomposed. Using the decomposed elements, (4a) and (4b) can be combined and written as the following system of equations:

$$\begin{bmatrix} Q_{\mathrm{LL}} & Q_{\mathrm{LU}} & Q_{\mathrm{LI}} & D_{\mathrm{ML}}^{\top} \\ Q_{\mathrm{UL}} & Q_{\mathrm{UU}} & Q_{\mathrm{UI}} & D_{\mathrm{MU}}^{\top} \\ Q_{\mathrm{IL}} & Q_{\mathrm{IU}} & Q_{\mathrm{II}} & D_{\mathrm{MI}}^{\top} \\ D_{\mathrm{ML}} & D_{\mathrm{MU}} & D_{\mathrm{MI}} & 0 \end{bmatrix} \begin{bmatrix} y_{\mathrm{L}} \\ y_{\mathrm{U}} \\ y_{\mathrm{I}} \\ \lambda \end{bmatrix} + \begin{bmatrix} \mu_{\mathrm{L}} \\ \mu_{\mathrm{U}} \\ \mu_{\mathrm{I}} \\ 0 \end{bmatrix} - \begin{bmatrix} \nu_{\mathrm{L}} \\ \nu_{\mathrm{U}} \\ \nu_{\mathrm{I}} \\ 0 \end{bmatrix} + \begin{bmatrix} d_{\mathrm{L}} \\ d_{\mathrm{U}} \\ d_{\mathrm{I}} \\ -\rho \end{bmatrix} = 0. \tag{5}$$

From the definitions of sets L and U, and complementary slackness in (4), we have:

$$y_{\mathrm{L}} = \underline{y}_{\mathrm{L}};\quad y_{\mathrm{U}} = \bar{y}_{\mathrm{U}};\quad \mu_{\mathrm{L}} = \nu_{\mathrm{U}} = \mu_{\mathrm{I}} = \nu_{\mathrm{I}} = 0. \tag{6}$$

Substituting the above in (5), we obtain the reduced KKT system:

$$\begin{bmatrix} Q_{\mathrm{LL}} \\ Q_{\mathrm{UL}} \\ Q_{\mathrm{IL}} \\ D_{\mathrm{ML}} \end{bmatrix} \underline{y}_{\mathrm{L}} + \begin{bmatrix} Q_{\mathrm{LU}} \\ Q_{\mathrm{UU}} \\ Q_{\mathrm{IU}} \\ D_{\mathrm{MU}} \end{bmatrix} \bar{y}_{\mathrm{U}} + \begin{bmatrix} Q_{\mathrm{LI}} & D_{\mathrm{ML}}^{\top} \\ Q_{\mathrm{UI}} & D_{\mathrm{MU}}^{\top} \\ Q_{\mathrm{II}} & D_{\mathrm{MI}}^{\top} \\ D_{\mathrm{MI}} & 0 \end{bmatrix} \begin{bmatrix} y_{\mathrm{I}} \\ \lambda \end{bmatrix} + \begin{bmatrix} -\nu_{\mathrm{L}} \\ \mu_{\mathrm{U}} \\ 0_{\mathrm{I}} \\ 0_{\mathrm{M}} \end{bmatrix} + \begin{bmatrix} d_{\mathrm{L}} \\ d_{\mathrm{U}} \\ d_{\mathrm{I}} \\ -\rho \end{bmatrix} = 0. \tag{7}$$

We can reduce the above system of equations to the following:

$$\begin{bmatrix} Q_{\mathrm{II}} & D_{\mathrm{MI}}^{\top} \\ D_{\mathrm{MI}} & 0 \end{bmatrix} \begin{bmatrix} y_{\mathrm{I}} \\ \lambda \end{bmatrix} = - \begin{bmatrix} Q_{\mathrm{IL}} \\ D_{\mathrm{ML}} \end{bmatrix} \underline{y}_{\mathrm{L}} - \begin{bmatrix} Q_{\mathrm{IU}} \\ D_{\mathrm{MU}} \end{bmatrix} \bar{y}_{\mathrm{U}} - \begin{bmatrix} d_{\mathrm{I}} \\ -\rho \end{bmatrix}, \tag{8a}$$

$$\begin{bmatrix} \nu_{\mathrm{L}} \\ \mu_{\mathrm{U}} \end{bmatrix} = \begin{bmatrix} Q_{\mathrm{LL}} \\ -Q_{\mathrm{UL}} \end{bmatrix} \underline{y}_{\mathrm{L}} + \begin{bmatrix} Q_{\mathrm{LU}} \\ -Q_{\mathrm{UU}} \end{bmatrix} \bar{y}_{\mathrm{U}} + \begin{bmatrix} Q_{\mathrm{LI}} & D_{\mathrm{ML}}^{\top} \\ -Q_{\mathrm{UI}} & -D_{\mathrm{MU}}^{\top} \end{bmatrix} \begin{bmatrix} y_{\mathrm{I}} \\ \lambda \end{bmatrix} + \begin{bmatrix} d_{\mathrm{L}} \\ -d_{\mathrm{U}} \end{bmatrix}. \tag{8b}$$

Note that the first system of equations can be used to solve for y_{I} and λ. Using this solution, we can solve the second system of equations for ν_{L} and μ_{U}. Studying the two systems of equations separately allows us to isolate the effect of changes to the right-hand side and upper-bound elements on the two sets of variables. We denote the inverse of the KKT matrix above by

$$B = \begin{bmatrix} Q_{\mathrm{II}} & D_{\mathrm{MI}}^{\top} \\ D_{\mathrm{MI}} & 0 \end{bmatrix}^{-1},$$

and define the auxiliary matrices:

$$W = -B \begin{bmatrix} Q_{\mathrm{IU}} & 0 \\ D_{\mathrm{MU}} & -I \end{bmatrix}, \quad T = - \begin{bmatrix} Q_{\mathrm{LU}} & 0 \\ -Q_{\mathrm{UU}} & 0 \end{bmatrix} + \begin{bmatrix} Q_{\mathrm{LI}} & D_{\mathrm{ML}}^{\top} \\ -Q_{\mathrm{UI}} & -D_{\mathrm{MU}}^{\top} \end{bmatrix} W. \tag{9}$$

Note that matrix B is well-defined because this problem is strictly convex and thus has a unique optimal solution. Therefore, when the partition is optimal after setting the value of active variables, the optimal solution remains unique. Consequently, the matrix B, which is the inverse of the KKT matrix of the new QP problem, is invertible.

Consider a problem with an arbitrary right-hand side and upper bound given by $(\rho', \bar{y}') = (\rho + \Delta\rho, \bar{y} + \Delta\bar{y})$. Henceforth, we will refer to the problem (2) with right-hand side and upper bound $(\rho, \bar{y})$ as the original problem and the one with right-hand side and upper bound $(\rho', \bar{y}')$ as the perturbed problem. We define $\Delta\rho = \rho' - \rho$ and $\Delta\bar{y} = \bar{y}' - \bar{y}$.

Using the partition $(\mathrm{L}, \mathrm{U}, \mathrm{I})$, we can construct a feasible solution for this perturbed problem. Specifically, the primal variables that belong to the active set can be obtained directly from the definition of the active set. The values of the dual variables corresponding to inactive primal variables can be determined through complementarity slackness. That is, $y'_{\mathrm{U}} = \bar{y}_{\mathrm{U}} + \Delta\bar{y}_{\mathrm{U}}$; $y'_{\mathrm{L}} = \underline{y}_{\mathrm{L}}$; $\mu'_{\mathrm{L}} = 0$; $\nu'_{\mathrm{U}} = 0$; $\mu'_{\mathrm{I}} = 0$ and $\nu'_{\mathrm{I}} = 0$. Using these and the matrices defined above in the reduced KKT system corresponding to $(\rho', \bar{y}')$ and some algebra, we obtain

$$\begin{bmatrix} y'_{\mathrm{I}} \\ \lambda' \end{bmatrix} = \underbrace{B\left(- \begin{bmatrix} Q_{\mathrm{IL}} \\ D_{\mathrm{ML}} \end{bmatrix} \underline{y}_{\mathrm{L}} - \begin{bmatrix} Q_{\mathrm{IU}} \\ D_{\mathrm{MU}} \end{bmatrix} \bar{y}_{\mathrm{U}} - \begin{bmatrix} d_{\mathrm{I}} \\ -\rho \end{bmatrix} \right)}_{=[y_{\mathrm{I}}, \lambda]^{\top}} + \underbrace{W \begin{bmatrix} \Delta\bar{y}_{\mathrm{U}} \\ \Delta\rho \end{bmatrix}}_{:=[\Delta y_{\mathrm{I}}, \Delta\lambda]}. \tag{10}$$

Substituting the expressions we obtain for y'_{I} and λ' in (8b), we obtain an analogous expression for the dual variables associated with the bounds of the perturbed problem

in terms of their counterparts in the original problem and the perturbation. Thus, we have:

$$\begin{bmatrix}\nu'_{\mathrm{L}}\\ \mu'_{\mathrm{U}}\end{bmatrix} = \underbrace{-\begin{bmatrix}-Q_{\mathrm{LL}} & -Q_{\mathrm{LU}}\\ Q_{\mathrm{UL}} & Q_{\mathrm{UU}}\end{bmatrix}\begin{bmatrix}\underline{y}_{\mathrm{L}}\\ \bar{y}_{\mathrm{U}}\end{bmatrix} - \begin{bmatrix}-Q_{\mathrm{LI}} & -D_{\mathrm{ML}}^{\top}\\ Q_{\mathrm{UI}} & D_{\mathrm{MU}}^{\top}\end{bmatrix}\begin{bmatrix}y_{\mathrm{I}}\\ \lambda\end{bmatrix} - \begin{bmatrix}-d_{\mathrm{L}}\\ d_{\mathrm{U}}\end{bmatrix}}_{=[\nu_{\mathrm{L}},\mu_{\mathrm{U}}]^{\top}} + \underbrace{T\begin{bmatrix}\Delta\bar{y}_{\mathrm{U}}\\ \Delta\rho\end{bmatrix}}_{:=[-\Delta\nu_{\mathrm{L}},\Delta\mu_{\mathrm{U}}]^{\top}},$$

where T is given in (9). Therefore, the solution to the perturbed problem can be decomposed into a solution of the original problem and a displacement term. The solutions of the perturbed problem are dual feasible if

$$\begin{bmatrix}\nu_{\mathrm{L}}\\ \mu_{\mathrm{U}}\end{bmatrix} + T\begin{bmatrix}\Delta\bar{y}_{\mathrm{U}}\\ \Delta\rho\end{bmatrix} \geq 0. \tag{11}$$

If $(\rho', \bar{y}')$ is such that $(\Delta\rho, \Delta\bar{y})$ further satisfies the primal feasibility condition

$$\underline{y}_{\mathrm{I}} \leq y_{\mathrm{I}} + W_{\mathrm{I(U+M)}}\begin{bmatrix}\Delta\bar{y}_{\mathrm{U}}\\ \Delta\rho\end{bmatrix} \leq \bar{y}_{\mathrm{I}} + \Delta\bar{y}_{\mathrm{I}}, \tag{12}$$

then the resulting solution for the perturbed problem satisfies first-order optimality conditions. The following result captures the consequence of the above development.

Proposition 1 *Let $P = (L, U, I)$ be the optimal partition of (2) for a given right-hand side and upper bound $(\rho, \bar{y})$. Consider $(\rho', \bar{y}') = (\rho + \Delta\rho, \bar{y} + \Delta\bar{y})$.*

(a) *There exists a polyhedron*

$$S_1 := \{(\rho', \bar{y}') \mid [\Delta\rho, \Delta\bar{y}]^{\top} \text{ satisfies (11)}\},$$

such that the partition P yields a solution (y, λ, μ, ν) that is feasible to the dual problem in (3) associated with any $(\rho', \bar{y}') \in S_1$.

(b) *Furthermore, there exists a polyhedron*

$$S_2 := \{(\rho', \bar{y}') \mid [\Delta\rho, \Delta\bar{y}]^{\top} \text{ satisfies (11) and (12)}\} \subseteq S_1,$$

such that P remains an optimal partition for (2) associated with any $(\rho', \bar{y}') \in S_2$.

We would like to emphasize that the property identified in Proposition 1 plays a crucial role in designing our algorithm. However, it is worth noting that our algorithm does not aim to explicitly construct the polyhedra S_1 and S_2. Instead, we generate metrics M, W, and T for every partition that the algorithm identifies. These matrices can be used to compute dual feasible solutions for any problem with a right-hand side and an upper bound $(\rho', \bar{y}') \in S_1$, by means of an affine transformation of the perturbation and the solution of the original problem.

Corollary 2 *Let P and (y, λ, ν, μ) be the optimal partition and optimal solution of problem (2) with right-hand side and upper bound $(\rho, \bar{y})$, respectively. Then, for any $(\rho', \bar{y}') \in S_1$, a feasible dual solution is derived from partition P as:*

$$\begin{bmatrix} y'_I \\ \lambda' \\ \nu'_L \\ \mu'_U \end{bmatrix} = \begin{bmatrix} y_I \\ \lambda \\ \nu_L \\ \mu_U \end{bmatrix} + \begin{bmatrix} W \\ T \end{bmatrix} \begin{bmatrix} \Delta \bar{y}_U \\ \Delta \rho \end{bmatrix}, \tag{13}$$

and $y'_U = \bar{y}_U + \Delta \bar{y}_U$; $y'_L = \underline{y}_L$; $\mu'_L = 0$; $\nu'_U = 0$; $\mu'_I = 0$; $\nu'_I = 0$.

If we solve problem (2) to optimality for a particular right-hand side $(\rho, \bar{y})$, then we have access to the corresponding optimal partition P. The above results indicate that this partition can be used to recover a solution for a perturbed program with right-hand side $(\rho', \bar{y}')$. If the solution satisfies (4c) and (4d), then we have a feasible dual solution. If the solution further satisfies bound constraints, then P serves as an optimal partition for the perturbed program. However, if either (4c) or (4d) is not met for the given right-hand side values $(\rho', \bar{y}')$, which is equivalent to $(\rho', \bar{y}') \notin S_1$, we recover a feasible dual solution by employing the PDAS method. We elaborate on the details in the next section.

2.2 Solution recovery using PDAS method

The PDAS method is a versatile algorithm used in various optimization problems, particularly those with inequality constraints. The PDAS methods provide an efficient approach to identifying optimal partitions for deterministic optimization problems [22, 24, 25, 27]. In these methods, the index sets are updated based on information available at the current solution. For our purpose, it suffices to identify a feasible dual solution as opposed to an optimal solution. Therefore, for a partition that yields an infeasible dual solution, we employ the PDAS method to recover a partition that yields a feasible dual solution. To achieve this, we develop four different versions of the PDAS method, as suggested in [23]. We begin by explaining how PDAS is employed, as described in [23], followed by a discussion of the four algorithmic variations considered in our investigation.

Algorithm 1 presents the PDAS method to find a feasible dual solution for a given right-hand side $(\rho', \bar{y}')$. This algorithm utilizes the initial partition $P^0 = (\mathrm{L}^0, \mathrm{U}^0, \mathrm{I}^0)$, and obtains its associated solution π^0 from equation (13). We proceed if this solution does not satisfy the dual feasibility condition for the second-stage problem. The sets V_P^0 and V_D^0 are determined based on the initial solution π^0, signifying the primal and dual variables that do not currently satisfy bound constraints and non-negativity constraints, respectively. Additionally, we initialize the iteration counter as $m = 0$.

At the beginning of iteration m, we first update the iteration counter $m = m + 1$ and select a set $V'_D \subseteq V_D^{m-1}$ and a subset $V'_P \subseteq V_P^{m-1}$. This selection is based on one of the four strategies we discuss later. Next, observe that variables within the subset V'_P violate either their upper or lower bounds. We partition V'_P into two distinct subsets: V'_{PL} that encompasses variables that are smaller than their lower bounds

and V'_{PU} that comprises variables that exceed their upper bounds. Similarly, observe that variables within the subset V'_D are either active at their lower or upper bound. We divide this subset into two separate subsets: V'_{DL} that consists of variables that are active at their lower bound and V'_{DU} that encompasses variables that are active at their upper bounds.

Following that, to modify the partition, we go through two phases, referred to as activation and inactivation phases. In the activation phase, we activate the elements in V'_{PL} at their lower bounds and the elements in V'_{PU} at their upper bounds. In the inactivation phase, we deactivate the variables associated with infeasible dual solutions. We achieve this by moving the elements of V'_D from U^{m-1} or L^{m-1} to the set I^m to obtain the updated index sets. We capture these updates in the following:

$$\mathrm{U}^m \leftarrow (\mathrm{U}^{m-1} \backslash V'_{DU}) \cup V'_{PU}, \tag{14a}$$

$$\mathrm{L}^m \leftarrow (\mathrm{L}^{m-1} \backslash V'_{DL}) \cup V'_{PL}, \tag{14b}$$

$$\mathrm{I}^m \leftarrow (\mathrm{I}^{m-1} \backslash V'_P) \cup V'_D \tag{14c}$$

1: **Input:** $P^0 = (\mathsf{L}^0, \mathsf{U}^0, \mathsf{I}^0)$.
2: **Initialization:** Compute π^0 using P^0; Identify V_P^0 and V_D^0; Set $m \leftarrow 0$.
3: **while** $V_D^m \neq \emptyset$ **do**
4: $m \leftarrow m + 1$.
5: Select subsets $V'_D \subseteq V_D^{m-1}$ and $V'_P \subseteq V_P^{m-1}$ from an elemental selection approach.
6: Identify subsets V'_{PL} and V'_{PU} from V'_P and subsets V'_{DL} and V'_{DU} from V'_D.
7: Perform the activation and inactivation phases to obtain the updated index sets as shown in (14a).
8: With the updated partition $P^m = (\mathsf{L}^m, \mathsf{U}^m, \mathsf{I}^m)$, solve the reduced KKT system to obtain π^m and V_P^m and V_D^m.
9: **end while**
10: **Return:** P^m and π^m.

Algorithm 1 PDAS

Finally, based on the modified partition $P^m = (\mathrm{L}^m, \mathrm{U}^m, \mathrm{I}^m)$, we solve the reduced KKT system (7) for $(\rho', \bar{y}')$ to achieve a new solution π^m. We also identify new sets of primal and dual infeasible variables, V_P^m and V_D^m, respectively. If the new solution is dual feasible, that is, $V_D^m \neq \emptyset$, then the method terminates. Otherwise, we repeat the above steps. In the context of the PDAS method, finite convergence is not generally guaranteed. Typically, previous works, e.g., [23] and [25], each impose restrictive assumptions on QP problems to ensure convergence. The paper [23] addresses this by examining problems constrained exclusively by the lower and upper bounds on decision variables. Likewise, the study [25] investigates scenarios where variables are exclusively bounded from above, a specific case of (1b). In this context, they impose rigorous conditions concerning the reduced Hessian regarding the upper-bounded variables. These conditions must hold for every potential partition to ensure convergence. Additionally, [24] discusses selection approaches that

involve defining auxiliary sets and solving reduced QP problems in each iteration of their procedure. While it is possible to ensure that such a procedure solves the actual QP problem (the recourse problem) to optimality, repeatedly solving auxiliary QP problems is computationally prohibitive in our setting. Moreover, the inexact procedures we introduce in the next section can operate using suboptimal dual solutions, suggesting that the procedure in [24] that recovers exact solutions may be superfluous. These studies collectively underscore the PDAS method's reliance on particular problem structures and assumptions for assured finite convergence.

Based on the insight we build from these studies, we introduce four approaches for selecting the subsets V'_P and V'_D:

- Single-Swap: Randomly choose an element $v_P \in V_P^{m-1}$ and an element $v_D \in V_D^{m-1}$. Set $V'_P \leftarrow \{v_P\}$ and $V'_D \leftarrow \{v_D\}$.
- Aggressive-Dual: Randomly select one element v_P from V_P^{m-1}, resulting in $V'_P \leftarrow \{v_P\}$, and set $V'_D \leftarrow V_D^{m-1}$.
- Aggressive-Primal: Randomly pick one element v_D from V_D^{m-1}, leading to $V'_D = \{v_D\}$, and set $V'_P \leftarrow V_P^{m-1}$.
- Aggressive-Primal-Dual: Set $V'_P \leftarrow V_P^{m-1}$ and $V'_D \leftarrow V_D^{m-1}$.

These approaches effectively showcase a range of options, spanning from aggressive to conservative. In Sect. 4, we undertake a comprehensive comparison of these four approaches to assess their performance in identifying a feasible dual solution. Our evaluation is based on criteria such as the frequency of encountered cycles, runtime efficiency, and the quality of the obtained solution.

3 Inexact proximal bundle algorithms for 2-SQP

In this section, we incorporate the PDAS method in decomposition-based proximal bundle algorithms for solving the 2-SQP problem in (1). For these algorithms, we make the following assumptions regarding (1).

(A1) The first-stage feasible region $\mathcal{X}$ is convex and compact.
(A2) The matrices P and Q are positive semidefinite and positive definite, respectively. Additionally, the matrix D is full rank.
(A3) The 2-SQP satisfies relatively complete recourse, and the recourse value is bounded from below by $L > -\infty$.
(A4) The randomness affects right-hand side elements or variable bounds, and the support Ω is a finite set.

Input: 2-SQP problem parameters, $\gamma \in (0,1)$, and $\sigma \in (0,1)$.
Initialization: $k \leftarrow 0$, $\mathcal{J}^0 = \{(L,0)\}$, $f^0(x) = \frac{1}{2}x^\top Px + c^\top x + L$, and $w^0 \in \mathcal{X}$.
while the stopping rule is not satisfied **do**
  $k \leftarrow k+1$
  Candidate update: identify a candidate solution:

$$x^k \in \arg\min\{f_\sigma^{k-1}(x) := f^{k-1}(x) + \frac{\sigma}{2}\|x - w^{k-1}\|^2 \mid x \in \mathcal{X}\}. \tag{15}$$

  Minorant generation: generate a minorant (α^k, β^k) using an exact, dual-based, or partition-based inexact procedure.
  Bundle update: update the bundle:

$$\mathcal{J}^k = \mathcal{J}^{k-1} \cup (\alpha^k, \beta^k).$$

  Approximation update: Obtain an updated approximation:

$$f^k(x) = \frac{1}{2}x^\top Px + c^\top x + \max_{(\alpha^j,\beta^j)\in\mathcal{J}^k}(\alpha^j + (\beta^j)^\top x). \tag{16}$$

  if $f^k(x^k) - f^k(w^{k-1}) \leq \gamma(f^{k-1}(x^k) - f^{k-1}(w^{k-1}))$ **then**
    Incumbent update: $w^k \leftarrow x^k$,
  else
    $w^k \leftarrow w^{k-1}$.
  end if
end while
Return: The optimal solution $x^* \leftarrow w^k$.

Algorithm 2 Proximal-bundle algorithm for 2-SQP

The exact proximal bundle algorithms have been well-studied for both deterministic and stochastic programs. In the case of 2-SP problems, the exact proximal bundle algorithm is commonly referred to as the regularized L-shaped method. The inexact variants we present here share the overall structure of the proximal bundle algorithms and differ only in how we generate the affine functions that constitute the bundle. We refer the reader to [28] or [29] for a detailed exposition of the proximal bundle algorithm applied to 2-SP problems. Here, we provide a brief overview of the algorithm and discuss some essential elements. We present the steps of a generic proximal bundle method in Algorithm 2. The exact variant and the two inexact variants introduced in this paper follow this generic pattern, differing only in how we compute the minorants. We present the procedure adopted to generate the minorants in Algorithms 3, 4, and 5 later in the section.

The bundle algorithms utilize a set of affine functions to approximate the expected recourse function, resulting in a piecewise affine convex outer approximation for the expectation-valued objective function in (1a). An algorithm iteration, indexed by k, begins by identifying a candidate solution x^k, which we obtain by solving a proximal

(or regularized) master problem (see Line 5). We construct a minorant at the candidate solution using either the exact or the inexact procedure, introduced later in this section. We denote by (α^k, β^k) the coefficients of the minorant that we construct in the current iteration. We add this minorant to the existing bundle to obtain the updated bundle (see Line 7). Using the updated bundle, we update the approximation of the first-stage objective function, which we subsequently use to update the incumbent solution in Line 9 by checking the following descent condition:

$$f^k(x^k) - f^k(w^{k-1}) \leq \gamma(f^{k-1}(x^k) - f^{k-1}(w^{k-1})). \tag{17}$$

The iterations in which the incumbent solution is updated are often referred to as the serious or descent steps. Otherwise, the iterations are referred to as the null steps, respectively. With the incumbent update step, an iteration of the algorithm is complete.

A few observations about the proximal bundle algorithm are in order. The construction of the minorants utilizes the dual solutions of the second-stage program. Let $\pi^k(\omega)$ be the optimal dual solution obtained by solving the second-stage program (1b) with input (x^k, ω). Due to weak duality, we have:

$$\begin{aligned} h(x,\omega) \geq\ & -\frac{1}{2}(y^k)^\top Q y^k + (\xi(\omega) - Cx)^\top \lambda^k + \underline{y}^\top \nu^k - \bar{y}^\top \mu^k \\ =\ & \underbrace{-\frac{1}{2}(y^k)^\top Q y^k + \xi(\omega)^\top \lambda^k + \underline{y}^\top \nu^k - \bar{y}^\top \mu^k}_{:=\alpha^k(\omega)} + \underbrace{\big(-(\lambda^k)^\top C\big)}_{:=\beta^k(\omega)} x,\ \forall x \in \mathcal{X}. \end{aligned} \tag{18}$$

The inequality above holds with equality when $x = x^k$ as a consequence of strong duality. In this case, we achieve an exact or supporting hyperplane for $h(x, \omega)$. However, when we use only a feasible solution, the inequality (18) still applies, but the resulting affine function may not be supporting even at $x = x^k$. Consequently, we obtain an inexact (lower-bounding) approximation.

Typically, when the bundle algorithms are applied for 2-SP, we solve all second-stage problems to optimality to compute the coefficients of the minorants as outlined in Algorithm 3. We refer to this procedure for generating minorants as the *exact procedure*.

1: **Input:** Candidate solution x^k and the sample set Ω.
2: **for** $\omega \in \Omega$ **do**
3: Solve the second-stage program and obtain the optimal solution π^k.
4: Calculate coefficients $(\alpha^k(\omega), \beta^k(\omega))$ from (18).
5: **end for**
6: Aggregate the coefficients of the affine function as:

$$\alpha^k = \sum_{\omega \in \Omega} p(\omega)\alpha^k(\omega), \quad \beta^k = \sum_{\omega \in \Omega} p(\omega)\beta^k(\omega). \tag{19}$$

return (α^k, β^k).

Algorithm 3 Exact procedure for generating minorants

Consequently, we obtain exact/supporting hyperplanes to individual recourse functions $h(x, \omega)$ and the resulting minorant satisfies $\alpha^k + (\beta^k)^\top x^k = \mathbb{E}[h(x^k, \tilde{\omega})]$, for every $k \geq 1$. Moreover, the approximation satisfies $f^k(x^k) = f(x^k)$ and $f^k(x) \leq f(x)$ for all $x \in \mathcal{X}$ and $k \geq 1$ (see (1a) and (16)). Over the iterations, the approximations are monotonically increasing, that is, $f^{k-1}(x) \leq f^k(x) \leq f^{k+1}(x) \leq \ldots \leq f(x)$ for all $x \in \mathcal{X}$. These properties play a vital role in showing the algorithm's convergence when we adopt the exact procedure for generating the minorants (see, for example, chapter 3 in [29]).

Secondly, the proximal bundle algorithm maintains two sequences of solutions: a candidate sequence $\{x^k\}$ and an incumbent sequence $\{w^k\} \subseteq \{x^k\}$. The criterion to identify the incumbent solutions plays a crucial role in the algorithm's convergence and computational performance. Notice that when we use the exact procedure, we can access the function value $f(x)$ at the candidate and incumbent solutions. This is because at the candidate solution we have $f^k(x^k) = f(x^k)$. Since the incumbent sequence is a subsequence of candidate solutions, we must have a $j < k$ such that $w^{k-1} = x^j$. Since we retain all the previously generated minorants in the bundle $\mathcal{J}^k$, we have $f^k(w^{k-1}) = f^j(x^j) = f(x^j)$. Consequently, the criterion we use to assess whether the current candidate solution can be accepted as an incumbent solution in (17) reduces to the descent condition: $f(x^k) - f(w^{k-1}) \leq \gamma(f^{k-1}(x^k) - f(w^{k-1}))$. The reader may be familiar with this descent criterion, as it is more commonly used in the proximal bundle algorithm (see, for example, [12]). However, assessing such a criterion is impossible when only inexact function values are available. Therefore, we assess a criterion using only the inexact function value obtained from the outer approximation f^k, which is computed at the current candidate and incumbent solutions. A similar criterion was previously employed in the stochastic decomposition algorithm, where coefficients of the minorants are stochastic; see [14].

Thirdly, how we manage the minorant bundle is also vital in determining the behavior of the algorithms. Notice that we add a new minorant to the bundle in every iteration (Step 7 of Algorithm 2). While we retain all previously generated minorants in the bundle, it is possible to apply bundle compression techniques wherein we retain only a subset of previously generated minorants. One such technique involves retaining only those minorants that contribute to the function approximation at the solutions where the assessment is being made, specifically, candidate and incum-

bent solutions. We assess this contribution by examining the optimal dual solutions, denoted by ζ^j, corresponding to an element $j \in \mathcal{J}^{k-1}$ obtained after solving (15). A bundle comprising of the active minorants at the latest candidate solution is thereby given as $\mathcal{J}^k = \{j \mid \zeta^j > 0\} \cup \{(\alpha^k, \beta^k)\} \cup \{(\hat{\alpha}, \hat{\beta})\}$ serves this purpose. Here, $(\hat{\alpha}, \hat{\beta})$ denotes the minorant generated at the incumbent solution, which was generated in an earlier iteration, that is, $(\hat{\alpha}, \hat{\beta}) = (\alpha^j, \beta^j)$ for some $j < k$. When we employ a bundle comprising only the active minorants, convergence analysis of the algorithms adds additional considerations. Since we focus on studying the consequences of introducing inexactness in minorant generation, we maintain a full bundle.

The advantages of employing the exact procedure come at the significant computational cost of solving second-stage programs for all observations in Ω. This is particularly true when dealing with large-scale problems in the second stage or when we have an extremely large number of possible scenarios. To alleviate this bottleneck, we introduce two inexact procedures for constructing the approximations used in the bundle algorithm. In these inexact procedures, we solve subproblems only for a subset of observations and use an appropriate approximate solution for the remaining observations. This results in inexactness both in the function value and the subgradient. Specifically, we can view our inexact procedures as specific manifestations of the inexact lower oracles. Moreover, in our algorithms, the level of inexactness is unknown but bounded and vanishes as the algorithm progresses. In this regard, the proposed inexact algorithms share some similarity with the general algorithm pattern for the inexact proximal bundle method presented in [13] (see Algorithm Pattern 4.2 in the cited reference). We elaborate on these similarities and key differences in Remark 3 later in this section after we present our algorithms.

3.1 Dual-based inexact procedure

Due to assumption (A4), the set of feasible dual solutions is not affected by either the first-stage decision variable or the observation (see (3)). We utilize this fundamental characteristic of the 2-SQP problems in the first inexact procedure for generating the minorants. We describe this procedure in Algorithm 4.

In the dual-based procedure, we maintain a collection of previously encountered dual solutions. At the beginning of iteration k, we denote this collection by Π^{k-1}. To generate a new minorant at candidate solution x^k, we select a subset Ω^k of observations and solve the subproblems corresponding to only observations $\omega \in \Omega^k$, i.e., with input (x^k, ω) in (1b). We append the optimal dual solutions obtained by solving these subproblems to the set Π^{k-1}, thus obtaining an updated set Π^k. For the remaining observations $\omega \in \Omega \backslash \Omega^k$, we obtain a suboptimal dual solution by utilizing an "argmax" step:

$$\pi(\omega) \in \arg\max\Big\{\frac{1}{2} y^\top Q y + (\xi(\omega) - Cx^k)^\top \lambda + \underline{y}^\top \nu - \bar{y}(\omega)^\top \mu \mid \pi \in \Pi^k\Big\}. \quad (20)$$

This step seeks to identify a solution from the set Π^k that yields the best objective function value for the dual of the second-stage problem. Next, by utilizing the

dual solutions, whether optimal or approximate, we compute the coefficients of the lower bounding minorant using the calculations shown in (18). When the dual-based inexact procedure is embedded in the proximal bundle algorithm in Algorithm 2, we initialize the collection $\Pi^0 = \emptyset$ and maintain the collection through the course of the algorithm. We refer to the algorithm as the dual-based proximal bundle algorithm.

Input: Candidate solution x^k, sample set Ω, collection of dual solutions Π^{k-1}.
Randomly generate a subset $\Omega^k \subset \Omega$.
for $\omega \in \Omega^k$ **do**
    Solve (1b) for (x^k, ω) and obtain the optimal dual solution $\pi(\omega)$.
    Calculate $(\alpha^k(\omega), \beta^k(\omega))$ from (18).
end for
Update the collection: $\Pi^k = \Pi^{k-1} \cup \{\pi(\omega) | \omega \in \Omega^k\}$
for $\omega \in \Omega \setminus \Omega^k$ **do**
    Using the argmax step in (20), identify an approximate dual solution.
    Calculate $(\alpha^k(\omega), \beta^k(\omega))$ from (18).
end for
Calculate the affine function coefficients from (19).
Return: (α^k, β^k) and Π^k.

Algorithm 4 Dual-based inexact procedure for generating minorant

The idea of reutilizing previously encountered solutions has been effectively utilized in SP. The earliest work that we are aware of is [30] for 2-SLP, where the author proposes bunching observations that lead to the same optimal basis of the second-stage linear programs and using these bunches to speed up the computations of minorant generation in the L-shaped method. Using a similar design principle, [31] proposes an inexact variant of the L-shaped method, and subsequently, [13] proposes an inexact proximal bundle method for 2-SLP. Reutilizing optimal dual solutions for 2-SLP with right-hand side uncertainty and the optimal dual basis for 2-SLP with random cost coefficients drives the computational efficiency of the stochastic decomposition algorithm; see [14] and [32], respectively.

3.2 Partition-based inexact procedure

While the dual-based inexact procedure reduces the computational effort required to solve the second-stage problems, it inadvertently leads to an escalation in the size of the dual set Π^k through successive iterations. This increase in size consequently renders the argmax step computationally burdensome, as it necessitates the examination of every element within the set Π^k during each iteration k. To avoid this drawback, we utilize the result of Proposition 1 to design an inexact procedure for generating the minorant in the proximal bundle algorithm. We summarize this procedure in Algorithm 5.

In each iteration, we solve second-stage problems for a subset of observations, as in Algorithm 4. However, instead of storing the dual solution, we store their optimal partitions. The input collection of partitions $\mathcal{P}^{k-1}$ is updated using partitions obtained by solving subproblems corresponding to $\omega \in \Omega^k$. For the remaining obser-

vations, we use the PDAS method described in Algorithm 1 to construct a feasible dual solution for all partitions $p \in \mathcal{P}^k$. We then invoke the argmax step in (20) to identify the best dual solution. Since there are finitely many partitions, the arg max step can be carried out computationally efficiently. We compute the lower bounding function coefficients using the calculations shown in (18). When the partition-based inexact procedure is embedded in the proximal bundle algorithm in Algorithm 2, we initialize the collection $\mathcal{P}^0 = \emptyset$ and consistently update this collection throughout the algorithm's execution. This algorithm is denoted as the partition-based proximal bundle algorithm.

Input: Candidate solution x_k, sample set Ω, collection of partitions $\mathcal{P}^{k-1}$.
Initialization: $\Pi^k \leftarrow \emptyset$.
Randomly generate a subset $\Omega^k \subset \Omega$
for $\omega \in \Omega^k$ **do**
    Solve (1b) for (x^k, ω) and obtain the optimal partition $P(\omega)$.
    Calculate $(\alpha(\omega), \beta(\omega))$ from (18).
end for
Update the collection: $\mathcal{P}^k \leftarrow \mathcal{P}^{k-1} \cup \{P(\omega) | \omega \in \Omega^k\}$.
for $\omega \in \Omega \setminus \Omega^k$ **do**
    Apply Algorithm 1 for all $p \in \mathcal{P}^k$ and gather all dual feasible solutions in a set defined as $\Pi^k = \Pi(x, \omega, \mathcal{P}^k)$.
    Use the argmax step in (20) to obtain an approximate dual solution.
    Calculate $(\alpha(\omega), \beta(\omega))$ from (18).
end for
Calculate affine function coefficients from (19).
Return: (α^k, β^k) and $\mathcal{P}^k$.

Algorithm 5 Partition-based inexact procedure for generating minorant

We close this section with the following remarks.

Remark 1 This remark addresses the process adopted to select the subset $\Omega^k \subseteq \Omega$. Let $\mathcal{J}_\omega \subseteq \{k = 0, 1, \ldots\}$ be the iterations, when $\omega \in \Omega^k$. The criterion for selecting Ω^k must satisfy $|\mathcal{J}_\omega| = \infty$. This criterion is equivalent to saying that for any $K > 0$, there exists a $j > K$ such that $j \in \mathcal{J}_\omega$. This requirement is satisfied, for instance, in a deterministic process such as the one adopted in the exact procedure where $\Omega^k = \Omega$. Another example is when observations are selected cyclically: $\omega_m \in \Omega^k$ for $k = n|\Omega| + m, n > 0, m = 1, \ldots, |\Omega|$. The requirement is also satisfied when randomization is adopted. For instance, when every observation is selected with fixed nonzero probability p, independently in each iteration. The requirement is satisfied due to the Borel-Cantelli lemma [33].

Remark 2 Like the proximal bundle algorithm, the level-decomposition method [15] provides an alternate approach to handle the challenges associated with the classical cutting-plane algorithm. Both methods rely on an outer approximation of the first-stage objective function and differ primarily in how a new candidate solu-

tion is identified. Our approaches for generating the piecewise affine functions for the outer approximations also apply to level-decomposition methods.

Remark 3 Our inexact algorithms largely fit into the general form of inexact proximal bundle methods previously studied, which was succinctly captured in Algorithm Pattern 4.2 of [12]. The minorant generating procedures in Algorithms 4 and 5 are specific manifestations of the inexact oracles. For SP problems with recourse, an effective design of inexact oracles is crucial in achieving the much-needed computational efficiency. As we shall see in the next sections, the inexact procedures provide asymptotic convergence guarantees when incorporated into the proximal bundle method and offer better numerical performance when compared to the exact variant. Our analysis will also show that our minorant generating procedures are asymptotically exact. We further note that both our inexact variants satisfy condition (*ia*) in Corollary 5.2 of [12]. Therefore, the so-called noise attenuation step identified in the general algorithm pattern is unnecessary for our algorithms. The critical distinguishing feature of our algorithms, relative to previous works on inexact proximal bundle methods, is the condition used to determine the descent step (17). For instance, in [11] and [10], a descent step is undertaken if $(\alpha^k + (\beta^k)^\top x^k) - (\alpha^j + (\beta^j)^\top w^{k-1}) \leq \gamma(f^{k-1}(x^k) - (\alpha^j + (\beta^j)^\top w^{k-1}))$ is satisfied. We refer the reader to [12] for other alternatives applied in inexact proximal bundle methods. Our choice of the descent condition is borrowed from the stochastic decomposition algorithm, and its application to the inexact proximal bundle method appears to be new.

3.3 Asymptotic convergence

In this section, we present the asymptotic convergence results for both variants of the inexact proximal bundle algorithm. Recall that $\{x^k\}$ and $\{w^k\}$ denote the sequence of candidate and incumbent solutions generated by the algorithm. The first result establishes the behavior of the first-stage function approximation along a converging sequence of solutions. Specifically, the result highlights a key feature of our dual-based and partition-based inexact procedures—they are, in fact, asymptotically exact.

Lemma 3 *Let $\mathcal{K}$ index an infinite subsequence of candidate solutions $\{x^k\}_{k=1}^{\infty}$ such that $\{x^k\}_{\mathcal{K}} \to \hat{x}$. Then have $\{f^k(x^k)\}_{\mathcal{K}} \to f(\hat{x})$.*

The analysis that leads to the above result is distinct when the procedure for generating the minorants differs. To establish the result in the case when we employ the dual-based procedure, we exploit the fact that the collection of feasible solutions exhibits the following property: $\Pi^k \subseteq \Pi^{k+1} \subseteq \ldots \subseteq \Pi$. This involves leveraging the continuity of the optimal value of (1) with respect to x. On the other hand, for the partition-based method, we rely on $\mathcal{P}^k \subseteq \mathcal{P}^{k+1} \subseteq \ldots \subseteq \mathcal{P}$ and the selection of the best solution obtained from applying PDAS through the argmax step. We establish the result based on Proposition 1, where we demonstrated that the optimal partition does not change when the solution x^k, and consequently, the right-hand side

$\rho = \xi(\omega) - Cx^k$ is within a polyhedron. The proof also establishes uniform convergence of the sequence $\{f^k\}$.

For the next results, the criterion used to update the incumbent solutions plays a vital role. We denote by $\mathcal{N} \subseteq \{0, 1, \ldots\}$ the iterations at which we update the incumbent (serious steps). We recognize a few quantities that appear frequently in our analysis. We denote by θ^k the optimal value of the first-stage approximate problem (15), i.e.,

$$\theta^k := f^{k-1}(x^k) + \frac{\sigma}{2}\|x^k - w^{k-1}\|^2.$$

We capture the perceived improvement at the candidate solution, relative to the current incumbent solution, by the term $\delta^k := f^{k-1}(w^{k-1}) - f^{k-1}(x^k)$. Since w^{k-1} is a feasible solution to the master problem, it follows that θ^k is bounded above by $f^{k-1}(w^{k-1})$, and in turn, by $f(w^{k-1})$. Optimality of x^k to the regularized master problem (15) implies that

$$\delta^k \geq \frac{\sigma}{2}||x^j - w^{k-1}|| \geq 0. \tag{21}$$

The next result establishes the asymptotic behavior of the sequence $\{\delta^k\}$.

Lemma 4 *There exists a subsequence $\mathcal{K}^* \subseteq \mathcal{N}$ such that $\{\delta^k\}_{\mathcal{K}^*} \to 0$.*

To prove the above result, we examine the case where the incumbent solution is updated a finite number of times, resulting in the entire sequence of δ^k converging to zero. We present this proof in "Appendix A" and focus on the second case, where the incumbent solution is updated infinitely, i.e., when $\mathcal{N}$ is infinite.

Proof From optimality we have for all $k_n \in \mathcal{N}$:

$$0 \leq \gamma\delta^{k_n} \leq f^{k_n-1}(w^{k_n-1}) - f^{k_n-1}(w^{k_n}) = f^{k_n-1}(w^{k_{n-1}}) - f^{k_n-1}(w^{k_n}).$$

The second inequality follows from $\gamma \in (0, 1)$ and the equality follows by noting that the incumbent solution is not updated during the null steps, implying $w^{k_n-1} = w^{k_{n-1}}$. The average improvement over M such iterations satisfies:

$$\frac{\gamma}{M}\sum_{m=1}^{M}\delta^{k_m} \leq \underbrace{\frac{1}{M}(f^{k_1-1}(w^{k_0}) - f^{k_M-1}(w^{k_M}))}_{(a)} + \frac{1}{M}\underbrace{\sum_{m=1}^{M}(f^{k_{(m+1)}-1}(w^{k_m}) - f^{k_m-1}(w^{k_m}))}_{(b)}$$

Under assumptions (A1) and (A3), part (a) converges to zero as $M \to \infty$. The term (b) also tends to zero, almost surely, due to the uniform convergence of the sequence f^k. Hence, we deduce:

$$0 \leq \lim_{k_m \in \mathcal{N}} \inf \delta^{k_m} \leq \lim_{k_m \in \mathcal{N}} \delta^{k_m} = \lim_{M \to \infty} \frac{1}{M} \sum_{m=1}^{M} \delta^{k_m} = 0.$$

The first equality follows from the fact that the subsequential limit is equal to its Cesàro limit. This leads to the conclusion that there exists a sequence, denoted as $\mathcal{K}^* \subseteq \mathcal{N}$, for which $\{\delta^k\}_{\mathcal{K}^*} \to 0$. □

While the above result establishes the existence of a subsequence, one can identify such a subsequence computationally. Notice from the above proof that we can construct an index set $\mathcal{K}^*$ using iterations k_m where $\delta^{k_m} \leq \frac{1}{M} \sum_{m=1}^{M} \delta^{k_m}$. We can establish that such an index set is an infinite set (as required in the proof), and every accumulation point of $\{\delta^k\}_{\mathcal{K}^*}$ is zero. In the following theorem, we use Lemma 3 and Lemma 4 to prove the convergence of $\{w^k\}$ to the optimal solution of (1).

Theorem 5 *Let $\{w^k\}_{k=1}^{\infty}$ denote the sequence of incumbent solutions. There exists a subsequence $\{w^k\}_{\mathcal{K}}$ for which every accumulation point is optimal to (1).*

Employing exact procedures to generate minorants has several advantages. From a computational point of view, it offers a direct means to verify the optimality of a given iterate. In the exact bundle method, we track $f(w^{k-1}) - \theta^k$ and terminate when this difference falls below a desired threshold. Both the quantities used to verify the termination criterion are readily available within the algorithm. We refer the reader to [29] for details. When inexact procedures are employed in proximal bundle methods to generate minorants, note that (21) and Lemma 4 ensure that there exists a subsequence of iterations, indexed by $\mathcal{K}$ such that $f^{k-1}(w^{k-1}) - \theta^k \to 0$ over $\mathcal{K}$. Although $f^{k-1}(w^{k-1}) - \theta^k$ is a suitable termination metric, stopping the algorithm when this quantity is lower than a threshold may be premature. While a thorough investigation of termination criteria is outside the scope of this paper, we identify a couple of criteria that can be employed when the partition-based inexact minorant-generating procedure is used. Firstly, we should expect the collection of partitions to stabilize. Secondly, the partitions identified for any observation, either by solving the subproblem or through the argmax procedure, must remain consistently the same. Designing termination criteria for when the dual-based procedure is employed is more challenging. Nevertheless, in our computational experiments, we run the algorithms for a minimum number of iterations and terminate when the function values at successive candidate solutions differ by no more than a certain threshold.

4 Numerical results

In this section, we present the results from the numerical experiments conducted on both the exact and inexact variants of the proximal bundle algorithm presented in this paper. We design our experiments to demonstrate the performance of the inexact proximal bundle algorithm compared to the exact proximal bundle algorithm. For the inexact algorithms, we explore their behavior when different fractions of subproblems are solved in every iteration. We also investigate the performance of these algorithms under varying levels of uncertainty. Before we report the results from these experiments, we share the experimental setup.

We conducted these experiments on instances of two problems encountered in power system planning and operations. The first problem, referred to as `PGP2`, is adapted from its original form and is based on data obtained from [34]. `PGP2` is a two-stage stochastic problem that optimizes power generation capacity increments through reserve facility installation. The goal is to minimize the total costs, including both installation and expected operating costs. This problem involves six variables and two constraints in the first stage and 23 variables and seven constraints in the second-stage problem. Of the six first-stage variables, four of them affect the second-stage optimization problem, thereby serving as the state variables. The random variables in the problem follow a discrete distribution.

The second problem is a modified IEEE-30 test system [35], a standard test system widely employed in power systems analysis. While the first problem is a power systems planning problem, `SODA30` concerns an operations problem known as economic dispatch. Economic dispatch refers to the process of optimizing power generation and distribution in a power system to meet demand efficiently. The system features 30 buses, 41 lines, six generators, and 21 loads. The renewable generator supply is considered random in our experiments. We refer to this problem as `SODA30`. It has a first stage involving 72 variables (of which 41 are state variables) and 98 constraints, while the second stage comprises 132 variables and 80 constraints. The random variables in the problem correspond to net demand and follow a truncated normal distribution.

For the computational experiments, we utilized a system with the Ubuntu 22.04.3 LTS operating system equipped with an Intel(R) Core(TM) i7-7600U CPU @ 2.80GHz, featuring a quad-core architecture with a maximum clock speed of 3.9 GHz and supported by 7.5 GB of RAM. All three algorithms were implemented using the C programming language. The optimization procedures were carried out using the Gurobi Optimizer (version 9.5), and the linear algebra routines were performed using LAPACK (Linear Algebra PACKage). Our implementation serves as a general-purpose solver for 2-SQP as it accepts input in SMPS file format [36]. Furthermore, we developed custom data structures that leverage the sparsity of the problems. The instances of the two problems used in our experiments, along with the algorithm source code, are available at the authors' GitHub repository (https://github.com/S MU-SODA/stochasticQP).

In our experiments, we used $\sigma = 1$. To obtain the value of L for the input of Algorithm 2, in many practical problems, the objective function of the second-stage problem is nonnegative, leading to the common practice of setting $L = 0$.

Conversely, if nonnegativity doesn't hold, such as in the SODA30 instances discussed here, we can employ the solution of the mean-value problem to generate an initial minorant $(\bar{\alpha}, \bar{\beta})$, which is then utilized in the initial bundle as $\mathcal{J}^0 = (\bar{\alpha}, \bar{\beta})$. We conduct 30 replications of all our experiments, each with randomly and independently generated instances of the above-mentioned problems. In presenting the results of the experiments, we report the metrics averaged over the 30 replications. We also report the standard deviations of the obtained metrics that are provided in parentheses adjacent to the corresponding results. The results for the partition-based inexact bundle algorithm utilize the aggressive-dual approach for selecting subsets in PDAS. We present a comparison with other approaches in Sect. 4.4 and justify our choice of the aggressive-dual approach.

4.1 Efficiency comparison of exact and inexact algorithms

The first experiment elucidates the comparative performance of the exact and inexact proximal bundle algorithms in different metrics. We note that the inexact methods that require noise attenuation (see [12]) are also suitable for comparison; such a comparison is beyond the scope of the current work. The results are presented in Table 1. The table shows the number of observations in the problem instance ($N = |\Omega|$), the total number of iterations before the stopping condition is satisfied (Iterations), the objective function value associated with the last incumbent solution (Obj. value), the total computational time (Time), which is reported in seconds, and the relative difference between the objective function value reported by the exact and inexact algorithms (Δ), and the absolute gap between the outer approximation and true objective function value (Gap) at the terminal solution. This gap, calculated as $f^k(w^k) - f(w^k)$, provides insight into the quality of the primal solution obtained by each algorithm, with smaller values indicating better solution quality. Note that the gap calculations are performed only for validation after the algorithm is terminated.

The optimal value derived from the exact algorithm closely approximates the true optimal value. Notice that as the value of N increases, the estimated objective function value tends to rise while the associated standard deviation tends to decrease. This behavior is consistent with the sample average approximation theory (see [37]). Although this pattern is evident in the results for PGP2, it appears somewhat more irregular in the case of SODA30. Recall that we solve only a random subset of subproblems generated independently in every iteration of the inexact procedures. In this experiment, we solve a subproblem associated with an observation with probability 0.1 in the inexact procedures. For both problems, the inexact methods achieve objective function values that are comparable to those obtained when the exact method is used. This is evident from the values of Δ in the table, which are less than 0.0145 and 0.0076 for PGP2 and SODA30, respectively.

We notice that, with comparable objective function values, the partition-based inexact bundle algorithm outperforms the exact algorithm in terms of the total computational time. For instance, when considering $N = 500$, the exact algorithm requires 11.20 seconds for the PGP2 instance and 37.71 seconds for the SODA30 instance. In contrast, the partition-based algorithm solves these instances in 5.53

Table 1 Numerical results for PGP2 and SODA30

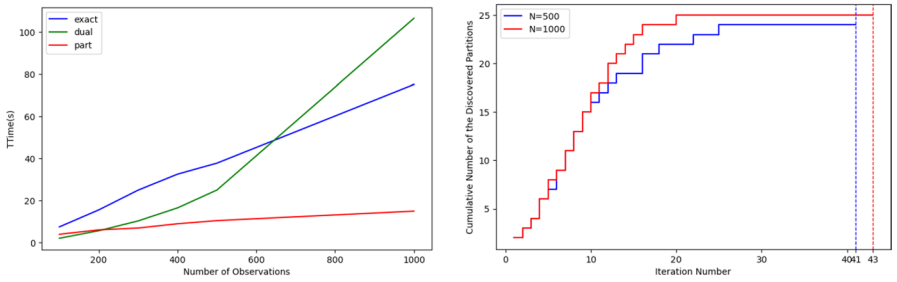
Alg	N	Iterations	Obj. value	Time(s)	Δ	Gap
PGP2						
E	100	43.16 (2.19)	556.97 (10.18)	1.84 (0.21)	–	0
	200	43.90 (3.43)	558.98 (8.20)	4.22 (0.42)	–	0
	300	43.86 (3.00)	559.04 (5.43)	6.79 (0.55)	–	0
	400	44.9 (3.12)	560.64 (5.12)	9.31 (0.76)	–	0
	500	44.66 (3.31)	560.44 (4.53)	11.20 (0.80)	–	0
	600	43.70 (2.96)	561.20 (3.74)	13.21 (1.02)	–	0
D	100	50.33 (7.51)	555.97 (10.37)	0.33 (0.06)	0.0018	0.40 (0.21)
	200	47.23 (7.08)	558.64 (8.16)	0.78 (0.19)	0.0006	0.14 (0.07)
	300	45.63 (4.15)	558.64 (5.38)	1.41 (0.19)	0.0008	0.10 (0.05)
	400	45.80 (3.95)	560.36 (5.21)	2.33 (0.30)	0.0005	0.08 (0.04)
	500	44.40 (4.03)	560.14 (4.55)	3.33 (0.53)	0.0005	0.06 (0.04)
	600	44.2 (3.73)	560.95 (3.81)	4.73 (0.73)	0.0006	0.06 (0.03)
P	100	50.20 (7.90)	548.86 (25.12)	0.74 (0.15)	0.0145	25.09 (43.93)
	200	47.16 (5.29)	558.32 (8.57)	1.46 (0.21)	0.0014	11.42 (47.66)
	300	44.93 (3.35)	558.48 (4.88)	2.45 (0.96)	0.0010	1.22 (3.13)
	400	47.76 (5.06)	560.42 (5.28)	4.70 (0.80)	0.0005	0.44 (1.31)
	500	45.20 (4.09)	560.32 (4.47)	5.53 (1.00)	0.0003	0.21 (0.91)
	600	45.86 (4.40)	561.01 (3.71)	7.77 (1.21)	0.0005	0.51 (1.31)
SODA30						
E	100	42.53 (2.72)	− 264,521 (141)	7.37 (0.51)	–	0
	200	43.40 (2.85)	− 264,529 (79)	15.46 (1.18)	–	0
	300	43.30 (2.64)	− 264,504 (89)	24.87 (1.75)	–	0
	400	43.86 (2.66)	− 264,490 (73)	32.48 (2.13)	–	0
	500	42.56 (2.60)	− 264,487 (71)	37.71 (2.56)	–	0
	1000	43.20 (2.24)	− 264,492 (49)	75.11 (4.30)	–	0
D	100	40.20 (0.48)	− 264,819 (1718)	1.97 (0.25)	0.0048	364.50 (333.57)
	200	40.30 (0.79)	− 264,937 (1337)	5.53 (0.41)	0.0041	755.30 (859.97)
	300	40.73 (2.27)	− 265,016 (1686)	10.21 (1.22)	0.0043	419.65 (327.95)
	400	40.43 (1.04)	− 265,430 (1814)	16.42 (0.99)	0.0051	1138.40 (1539.03)
	500	40.73 (1.25)	− 264,775 (1297)	24.92 (1.90)	0.0036	2206.37 (5463.31)
	1000	40.40 (0.85)	− 265,202 (3332)	106.51 (6.83)	0.0076	4861.10 (7578.75)
P	100	44.43 (3.23)	− 264,520 (141)	3.83 (1.04)	< 10e−4	0.78 (3.47)
	200	44.06 (3.62)	− 264,530 (78)	5.96 (2.31)	< 10e−4	0.55 (2.23)
	300	43.06 (3.23)	− 264,505 (90)	6.83 (0.84)	< 10e−4	0.50 (1.70)
	400	43.30 (2.43)	− 264,491 (73)	8.85 (1.30)	< 10e−4	0.12 (0.14)
	500	43.26 (2.55)	− 264,487 (70)	10.36 (0.71)	< 10e−4	0.23 (0.25)
	1000	42.9 (2.78)	− 264,493 (49)	14.87 (1.50)	< 10e−4	0.13 (0.14)

The algorithm names in the first column are abbreviated as **E**xact, **D**ual-based, and **P**artition-based

seconds for PGP2 and 10.36 seconds for SODA30. Since the number of iterations is comparable in the two algorithms, this observation is attributed to the per-iteration computational effort required to solve subproblems for all observations, as compared to using the PDAS method and argmax step. We emphasize the role of carefully designed data structures and subroutines in implementing the PDAS method and argmax step, which contribute to achieving this computational performance improvement.

In addition to the comparable objective function values, we also analyze the quality of the primal solutions obtained by comparing the gap between the dual bound $f^k(w^k)$ and the true objective function value $f(w^k)$. For the PGP2 instances, the Dual-based algorithm shows consistently small gaps (ranging from 0.40 to 0.06) that decrease as N increases, indicating improved solution quality for larger problem sizes. The Partition-based algorithm initially shows larger gaps for smaller PGP2 instances (25.09 at $N = 100$), but these quickly diminish as N increases (to 0.51 at $N = 600$). For the SODA30 instances, we observe a contrasting pattern: the Dual-based algorithm exhibits larger gaps that increase with problem size (from 364.50 to 4861.10), while the Partition-based algorithm maintains remarkably small gaps (all below 0.78) across all problem sizes. Note that we report the absolute gap in the last column of Table 1, unlike the relative value reported for Δ. Therefore, even the dual-based algorithm provides a solution whose value is within 2% of optimality, even under the worst case (SODA30 instances with $N = 1000$). Nevertheless, this analysis reveals that the Partition-based algorithm offers computational advantages and produces high-quality primal solutions, particularly for the large SODA30 instances, where the Dual-based algorithm struggles to provide tight bounds between the dual and primal solutions.

The dual-based inexact bundle algorithm exhibits the best performance in terms of the total computational time in PGP2. In SODA30, while it has the best performance when $N \leq 200$, the dual-based algorithm shows the poorest performance when N exceeds 200 (see Fig. 1a). This decline in performance is attributed to the substantial expenses associated with the resource-intensive argmax step (20) in the dual-based algorithm, which necessitates an exhaustive search across all acquired dual solutions. To substantiate this point, we assessed the sizes of the sets Π and $\mathcal{P}$ in the largest instances of PGP2 and SODA30 (with 600 and 1000 observations, respectively). For the dual-based algorithm, the sizes were 2665 and 4075 for PGP2 and SODA30, respectively. In contrast, for the partition-based algorithm, the sizes were 270 and 28. This disparity elucidates why the performance of the dual-based algorithm is significantly poorer for SODA30. As the problem size grows, we expect the set Π to enlarge as observed when transitioning from PGP2 to SODA30; thereby increasing the computational cost of the argmax step. However, it is still possible for the set $\mathcal{P}$ to remain within a reasonable size. While it is impossible to predict the number of parti-



(a) Comparison of Computational Time (b) Partition Discovery Pattern

Fig. 1 Comparison of efficiency of proximal bundle methods

tions one may encounter, based on our empirical observation, we expect the partition-based algorithm to outperform the dual-based algorithm in large-scale settings.

Figure 1b illustrates the progress of the partition-based algorithm (number of iterations along the horizontal axis) in terms of discovering new partitions. The vertical axis shows the cumulative number of partitions discovered. The figure illustrates the progress in two instances of SODA30 with 500 and 1000 observations, respectively. In the case of 500 observations, the total number of iterations amounted to 41. Notably, the latest iteration wherein a new partition emerged from solving second-stage problems was iteration 25, resulting in a total of 24 identified partitions. In the case of 1000 observations, the last new partition was discovered in iteration 20, and the algorithm ran for 43 iterations. Recall that the convergence analysis of the partition-based algorithm relies on encountering the same partition after a finite number of iterations for any observation. In other words, we expect that all relevant partitions are discovered after a certain number of iterations. The figure provides empirical evidence of this fact.

4.2 Impact of sampling fraction on partition-based

As previously discussed in Remark 1, the asymptotic performance of the algorithm is achieved under different subset selection criteria. Since the per-iteration computational cost of inexact algorithms is proportional to the number of subproblems solved (size of $|\Omega^k|$), our next experiment assesses the impact of the selection strategy. We report the results only for the partition-based algorithm. One criterion that fulfills the necessary conditions for the selection is to sample each observation with a constant nonzero probability, denoted as p, in every iteration. We examined the impact of different p values on a crucial performance metric: the computational time before termination. These results are presented in Fig. 2 for instances of SODA30 with different numbers of observations. The figure shows the computational time (vertical axis) for different values of p between 0.01 and 1 along the horizontal axis. Notice that $p = 1$ corresponds to the exact method.

The figure demonstrates that as p values increase, computational time initially decreases, reaching a minimum value at either $p = 0.05$ or $p = 0.10$ for all instances. However, once p values exceed this point, the computational time increases. The higher computational cost for very small p values is attributed to weaker minorants that prolong the algorithm's termination. Similarly, for large p values, the increased computational cost is because of the need to solve a larger number of second-stage QP problems. Nevertheless, these results provide empirical evidence that high-quality solutions can be obtained even by solving a small fraction of subproblems.

4.3 Effect of variance on the number of discovered partitions

In this experiment, we investigate how the variance of random variables (on the right-hand sides) influences the performance of the partition-based algorithm. Specifically, we monitor the number of identified partitions. We report the results for the SODA30 instance with 1000 observations in Table 2. We vary the variance of two random variables, both with mean one, between 0.01 and 4. We applied

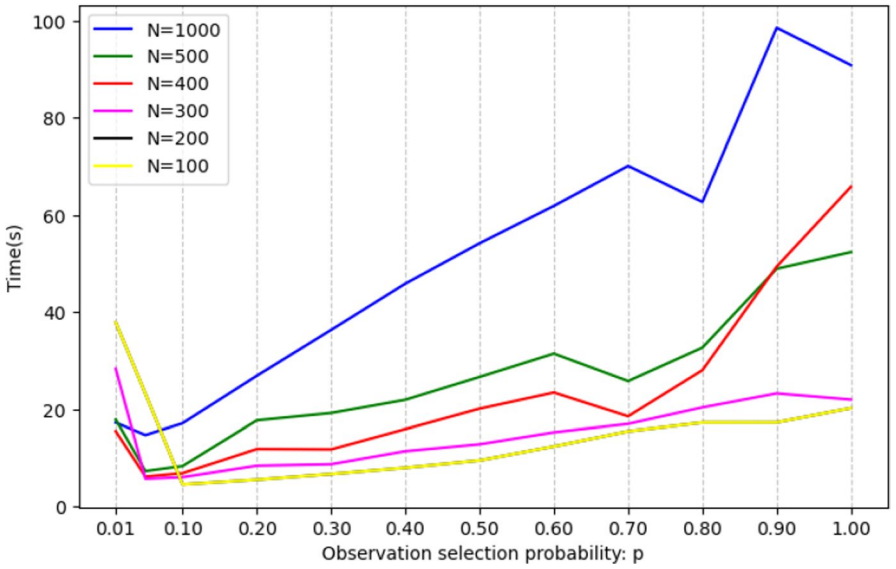


Fig. 2 Impact of sampling fraction on partition-based method (instances of SODA30)

Table 2 Effect of variance on discovered partitions

Variance	0.01	0.10	0.50	1.00	1.50	2.00	2.50	3.00	4.00
# Partitions	3.19	7.83	8.34	8.30	8.99	8.18	11.47	11.06	11.73
Time(s)	7.5	15.4	24.7	25.2	26.9	28.0	30.7	31.7	33.6

the partition-based algorithm with a sample fraction of 0.1 to solve these problem instances.

The table shows the total computational time and the number of discovered partitions for different variance values. The results show that as the variability on the right-hand side amplifies, it leads to heightened variability in the second-stage subproblems. Consequently, in line with expectations, there is an increase in the number of identified partitions. This increases the computational effort in PDAS and the argmax step, which is aggregated in the computational time mentioned in the table.

4.4 Comparison of different versions of implementation of PDAS

In our last experiment, we compared the different subset selection approaches within PDAS, namely Single-Swap, Aggressive-Dual, Aggressive-Primal, and Aggressive-Primal-Dual. For this experiment, we use a deterministic instance of SODA30 (a QP problem with linear constraints). Beginning with the same random partition as input, we run the PDAS method until it discovers an optimal solution. Additionally, we note the results when the first feasible dual solution is discovered. We conduct 30 independent replications of this experiment and report the results in Table 3. In this table, we report the range of relative gaps between the optimal objective function value and the objective function at the solutions reported from

Table 3 Numerical results for PDAS

Approach	Dual feasibility			Optimality		
	Time(s)	Gap	# Cycles	Time(s)	Gap	# Cycles
Single-swap	5.77×10^{-6}	[0.04,294.27]	3	4.90×10^{-6}	0.00	10
Aggressive-dual	6.00×10^{-6}	[145.29,295.86]	0	5.08×10^{-6}	0.00	17
Aggressive-primal	6.27×10^{-6}	[246.73 ,295.86]	12	–	–	30
Aggressive-primal-dual	5.95×10^{-6}	[26.76,295.86]	8	–	–	30

PDAS. We observed cycles with all four subset selection approaches while looking for the optimal solution. In the PDAS method, a cycle occurs if we encounter the same partition as one obtained in a previous iteration. We document the number of instances where the algorithm encounters cycles (# Cycles). Note that the average computational time and the range of relative gaps are computed only for replications where no cycles were encountered.

Analysis of results reported in Table 3 reveals that the runtimes of the four algorithms are closely comparable. The Single-Swap approach yields the best objective function values for all instances where no cycles were encountered, with the smallest relative gap of 0.04. This suggests the potential for achieving a high-quality, objective function value with the Single-Swap approach compared to other approaches. Single-Swap also exhibits the lowest occurrence of cycles when searching for the optimal solution. Notably, the Aggressive-Dual approach consistently avoids encountering cycles while searching for a feasible dual solution. This observation aligns with our expectations, as the Aggressive-Dual approach, in its pursuit of a feasible dual solution, strives to push the solution into the dual feasible space without considering primal feasibility. This approach also demonstrated better performance in terms of the number of times it encountered cycles while searching for optimal solutions. Both the Single-Swap and Aggressive-Dual approaches resulted in an optimality gap of 0.00 in all instances they could solve. In contrast, we encountered cycles when solving to optimality in all instances with both Aggressive-Primal and Aggressive-Primal-Dual.

We note that the additional time required to discover an optimal dual solution using Single-Swap and Aggressive-Dual approaches is negligible in replication, where no cycles are encountered. However, both methods encounter more cycles when pursuing the optimal dual solution. For instance, we encounter cycles in seven additional replications when attempting to discover the optimal dual solution after a feasible solution has been obtained. Fortunately, we only need to ensure that the PDAS method yields a feasible dual solution when applied in the partition-based proximal bundle algorithm. The results from this experiment, especially the absence of cycles, provide empirical justification for using the Aggressive-Dual approach in implementing the PDAS method within the partition-based proximal bundle algorithm. PDAS algorithm designs that ensure finite convergence or the absence of cycles without restrictive assumptions are an open research question that is beyond the scope of this paper. Therefore, we currently rely only on empirical evidence.

5 Conclusions

In this paper, we examine the impact of varying the right-hand side on the optimal partition of a QP problem. To compute the solution of the associated QP problem for a given partition, we introduced a linear system derived from the reduced KKT system. Subsequently, we presented four distinct implementations of the PDAS method, specifically tailored to project an infeasible dual solution of the QP problem into the feasible region. We proposed two variations of the inexact proximal bundle algorithm – the dual-based and partition-based – customized for 2-SQP problems. A key feature of these algorithms is their ability to guarantee global convergence while accommodating imprecision in solving second-stage problems during the minorant generation process within the proximal bundle algorithm. The numerical experiments comparing the inexact algorithms with the exact proximal bundle algorithm demonstrate that the partition-based algorithm outperforms the exact algorithm in total computational time while delivering a solution of similar quality. While our numerical results were conducted on multiple instances of two problems, experimentation on a larger array of problems will further underscore the efficiency of the inexact procedures. Incorporating bundle management/compression techniques, such as those discussed at the beginning of Sect. 3, is a potential algorithmic improvement we will pursue in our future research. The dual-based algorithm exhibits less favorable performance, particularly when the sample size increases. Development of reliable stopping rules, convergence-rate analysis, conducting more comprehensive numerical experiments, and extending the inexact procedures to multistage SQP are fruitful future directions that we intend to pursue.

Appendix A: Omitted proofs

Proof of Lemma 3 Consider the function $f^k(x)$, composed of two distinct parts. The first part is a polynomial function given by $\frac{1}{2}x^\top Px + c^\top x$. Due to the inherent continuity of polynomial functions, it follows that $\frac{1}{2}(x^k)^\top Px^k + c^\top x^k \to \frac{1}{2}\hat{x}^\top P\hat{x} + c^\top \hat{x}$, whenever the sequence $\{x^k\} \to \hat{x}$. We next establish the convergence of the second part of $f^k(x)$, i.e., $\max_{j\in J^k}(\alpha^j + (\beta^j)^\top x^k) \to \mathbb{E}[h(\hat{x}, \tilde{\omega})]$. In this regard, from Proposition 2.21 of [37], note that the $h(x, \omega)$ is a continuous function. Now, let us define $\mathcal{J}_\omega$ as the sequence of iterations when the subproblem associated with ω is solved exactly. We consider the rest of the proof for the dual-based and partition-based inexact bundle algorithms distinctly.

Dual-based algorithm: In the dual-based algorithm, we have $\Pi^k \subseteq \Pi^{k+1} \subseteq \ldots \subseteq \Pi$, and the argmax step utilizes this collection of dual solutions. Therefore, if we define $h^k(x, \omega) = \max\left\{\frac{1}{2}y^\top Qy + (\xi(\omega) - Cx)^\top \lambda + \underline{y}^T\nu - \bar{y}(\omega)^\top \mu \mid \pi \in \Pi^k\right\}$, we observe the following relationship for all $\omega \in \Omega$:

$$h^k(x, \omega) \leq h^{k+1}(x, \omega) \leq \ldots \leq h(x, \omega). \tag{A1}$$

Since $\mathcal{X}$ is compact, the above monotonic sequence of continuous functions converges uniformly. Following the continuity of h and convergence of $\{x^k\}_{\mathcal{K}}$, we have for every $\epsilon > 0$ there exists $K < \infty$, such that for $j \in \mathcal{J}_\omega \cap \mathcal{K}$ and $j > K$, we have:

$$|h(\hat{x},\omega) - h^j(x^j,\omega)| = |h(\hat{x},\omega) - h(x^j,\omega)| < \frac{\epsilon}{3}. \tag{A2}$$

The equality is because we solved the subproblem exactly. Since h^j is a continuous function, there exists $K' < \infty$ such that:

$$|h^j(x^j,\omega) - h^j(x^k,\omega)| < \frac{\epsilon}{3},$$

for all $k, j \in \mathcal{K}$ and $k, j > K'$. From uniform convergence of $\{h^k\}$, there exists $K'' < \infty$ such that:

$$|h^j(x^k,\omega) - h^k(x^k,\omega)| < \frac{\epsilon}{3}, \tag{A3}$$

for every $k, j > K''$. Thus, for $j \in \mathcal{J}_\omega$ and $k \in \mathcal{K}$ such that $k > j > \max(K, K', K'')$, we have

$$\begin{aligned}|h(\hat{x},\omega) - h^k(x^k,\omega)| \leq\ & |h(\hat{x},\omega) - h^j(x^j,\omega)| \\ & + |h^j(x^j,\omega) - h^j(x^k,\omega)| + |h^j(x^k,\omega) - h^k(x^k,\omega)| < \epsilon.\end{aligned}$$

The sampling procedure guarantees the existence of a j_ω that satisfies the above requirement for all $\omega \in \Omega$. Therefore, $\left| \sum_{\omega\in\Omega} p(\omega)(h(\hat{x},\omega) - \left(\alpha^k(\omega) + \beta^k(\omega)x^k\right) \right| \leq \epsilon$, for $k > \max_{\omega\in\Omega}\{j_\omega\}$. We have used $h^k(x^k,\omega) = \alpha^k(\omega) + \beta^k(\omega)x^k$ in the above. The result is thus established.

Partition-based algorithm: Here, we define $h^k(x,\omega) = \max\left\{\frac{1}{2}y^\top Qy + (\xi(\omega) - Cx)^\top\lambda + \underline{y}^T\nu - \bar{y}(\omega)^\top\mu \ |\pi \in \Pi(x,\omega,\mathcal{P}^k)\right\}$. We observe that a similar relationship as (A1) holds in the partition-based method since $\mathcal{P}^k \subseteq \mathcal{P}^{k+1} \subseteq \ldots \subseteq \mathcal{P}$ and the argmax procedure uses the collection of partitions. For every $\epsilon > 0$, let us define parameter K as (A2). As a consequence of Proposition 1; there exists a ball with radius $\delta(\omega)$ centered at $\hat{x}$ such that the optimal partition is the same for $x \in \{\chi| \ ||\chi - \hat{x}|| \leq \delta(\omega)\}$ for every $\omega \in \Omega$. Finally, since x^k converges to $\hat{x}$, there exists a K' such that whenever $k > K'$, we have $||x^k - \hat{x}|| < \delta(\omega)$ for all $\omega \in \Omega$. The sampling procedure ensures that we have a $j_\omega \in \mathcal{J}_\omega$ and $j_\omega > \max(K, K')$. Therefore, we have

$$|h(\hat{x},\omega) - h(x^k,\omega)| = |h(\hat{x},\omega) - h^k(x^k,\omega)| \leq \epsilon. \tag{A4}$$

for all $k > j_\omega$. The rest of the proof follows as in the case for the dual-based method. □

Proof of Lemma 4 We study two possible cases depending on whether $\mathcal{N}$ is finite or infinite. Here, we present the proof under the case when $\mathcal{N}$ is finite, there exists a $K < \infty$ such that for all $k > K$, we have $f^k(x^k) > f^k(\bar{w}) + \gamma(f^{k-1}(x^k) - f^{k-1}(\bar{w}))$, and $w^{k-1} = \bar{w}$. Since $f^k(\bar{w}) \geq f^{k-1}(\bar{w})$

$$f^k(x^k) > f^{k-1}(x^k) + (1-\gamma)[f^{k-1}(\bar{w}) - f^{k-1}(x^k)].$$

Let $g^k \in \partial f^k(x^k)$. From the subgradient inequality and the above inequality, we have:

$$f^k(x) \geq f^k(x^k) + \langle g^k, x - x^k\rangle \geq f^{k-1}(x^k) + (1-\gamma)\delta^k + \langle g^k, x - x^k\rangle. \tag{A5a}$$

Since x^k is the minimizer of (15), we have $0 \in \partial f^{k-1}(x^k) + \sigma(x^k - \bar{w})$. Again, from subgradient inequality, we have:

$$f^k(x) \geq f^{k-1}(x) \geq f^{k-1}(x^k) + \sigma\langle(\bar{w} - x^k), (x - x^k)\rangle. \tag{A5b}$$

Thus, from (A5a) and (A5b), we have:

$$\begin{aligned}
f^k(x) &\geq \max\{f^{k-1}(x^k) + (1-\gamma)\delta^k + \langle g^k, x - x^k\rangle, f^{k-1}(x^k) + \sigma\langle(\bar{w} - x^k), (x - x^k)\rangle\} \\
&= f^{k-1}(x^k) + \sigma\langle\bar{w} - x^k, x - x^k\rangle + \max\{(1-\gamma)\delta^k + \langle g^k - \sigma(\bar{w} - x^k), x - x^k\rangle, 0\}. \\
\Rightarrow f^k(x) + \frac{\sigma}{2}||x - \bar{w}||^2 &\geq f^{k-1}(x^k) + \frac{\sigma}{2}||(x^k - \bar{w}) - (x^k - x)||^2 + \\
&\quad \sigma\langle\bar{w} - x^k, x - x^k\rangle + \max\{(1-\gamma)\delta^k + \langle g^k - \sigma(\bar{w} - x^k), x - x^k\rangle, 0\} \\
&= f^{k-1}(x^k) + \frac{\sigma}{2}||x^k - \bar{w}||^2 + \frac{\sigma}{2}||x^k - x||^2 + \\
&\quad \max\{(1-\gamma)\delta^k + \langle g^k - \sigma(\bar{w} - x^k), x - x^k\rangle, 0\}.
\end{aligned}$$

Specifically by setting $x = x^{k+1}$, we have

$$\theta^{k+1} - \theta^k \geq \frac{\sigma}{2}||x^k - x^{k+1}||^2 + \max\{(1-\gamma)\delta^k + \langle g^k - \sigma(\bar{w} - x^k), x^{k+1} - x^k\rangle, 0\}.$$

Therefore, the sequence $\{\theta^k\}$ is nondecreasing and bounded above by $f(\bar{w})$. Therefore, we must have $\theta^{k+1} - \theta^k \to 0$. More generally, noting that $\|g^k\| \leq G$ and defining $\Delta x = ||x - x^k||$, we have

$$\begin{aligned}
\theta^{k+1} - \theta^k &\geq \min_{\Delta x}\left\{\frac{\sigma(\Delta x)^2}{2} + \max\{(1-\gamma)\delta^k - 2G\Delta x, 0\}\right\} \\
&\geq \begin{cases} \frac{\sigma(1-\gamma)^2(\delta^k)^2}{8G^2} & \text{if } (1-\gamma)\delta^k \leq \frac{4G^2}{\sigma} \\ \frac{(1-\gamma)\delta^k}{2} & \text{otherwise,} \end{cases} \\
&\geq 0.
\end{aligned}$$

The last inequality follows because $\delta^k \geq 0$. Since $\theta^{k+1} - \theta^k \to 0$, we conclude that $\delta^k \to 0$ when $\mathcal{N}$ is finite. □

Proof of Theorem 5 From Lemma 4, we have a sequence $\mathcal{K}$ such that $\{\delta^k\}_{\mathcal{K}} \to 0$. Thus, assumption (A1) and $0 \leq \frac{\sigma}{2}||w^k - w^{k-1}|| \leq \delta^k$ ensure the existence of an accumulation point $\bar{w} \in \mathcal{X}$. From Lemma 3, we have $\lim_{k\in\mathcal{K}} f^{k-1}(w^k) = f(\bar{w})$. Combining the two, we have

$$\lim_{k\in\mathcal{K}} \theta^k = \lim_{k\in\mathcal{K}} f^{k-1}(w^k) + \frac{\sigma}{2}||w^k - w^{k-1}|| = f(\bar{w}).$$

Let's assume $\bar{w}$ is not optimal. Let $f_\sigma(w)$ denote the Moreau–Yosida regularization of f given by $f_\sigma(w) := \min_{x\in\mathcal{X}} \left\{ f(x) + \frac{\sigma}{2}\|x - w\|^2 \right\}$. By Lemma 8 of [29], we have $f_\sigma(w^{k-1}) < f(w^{k-1})$. Since $f^{k-1}(x) \leq f(x)$ for all $x \in \mathcal{X}$, we have $\theta^k \leq f_\sigma(w^{k-1})$. So, we have $f(\bar{w}) = \lim_{k\in\mathcal{K}} \theta^k < \lim_{k\to\infty} f(w^{k-1}) = f(\bar{w})$, which leads to a contradiction.

Acknowledgements We thank the coordinating editor and the two anonymous referees for their detailed reading of the paper. Their suggestions helped us improve the paper's presentation to make it more compelling.

Funding Open access funding provided by SCELC, Statewide California Electronic Library Consortium. The Office of Naval Research supported this work under Grant Number N00014-22-1-2603. Additionally, the first author acknowledges the Computational Science and Engineering Fellowship awarded by the Center for Research Computing and the SMU Moody School of Graduate and Advanced Studies. Their support has been invaluable in enabling the completion of this research.

Data availability The source code and details of the instances used in the numerical experiments are available in the GitHub repository (https://github.com/SMU-SODA/stochasticQP).

Declarations

Conflict of interest The authors declare they have no financial or non-financial interests to disclose.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Wets, R.R.-J.: Programming under uncertainty: the equivalent convex program. SIAM J. Appl. Math. **14**(1), 89–105 (1966)

2. Rockafellar, R.T., Wets, R.-J.: Scenarios and policy aggregation in optimization under uncertainty. Math. Oper. Res. **16**(1), 119–147 (1991)
3. Rockafellar, R.T., Preékopa, A., Wets, R.J.-B., Wets, R.J.-B. (eds.): A Lagrangian Finite Generation Technique for Solving Linear-Quadratic Problems in Stochastic Programming, pp. 63–93. Springer, Berlin (1986)
4. Chen, X., Qi, L., Womersley, R.S.: Newton's method for quadratic stochastic programs with recourse. J. Comput. Appl. Math. **60**(1–2), 29–46 (1995)
5. Mehrotra, S., Ozevin, M.G.: Decomposition based interior point methods for two-stage stochastic convex quadratic programs with recourse. Oper. Res. **57**(4), 964–974 (2009)
6. Birge, J.R., Qi, L., Chen, X.: A stochastic newton method for stochastic quadratic programs with recourse. Technical report AM94/9, School of Mathematics, University of New South Wales (1995)
7. Liu, J., Sen, S.: Asymptotic results of stochastic decomposition for two-stage stochastic quadratic programming. SIAM J. Optim. **30**(1), 823–852 (2020)
8. Lemaréchal, C.: Nonsmooth optimization and descent methods. Technical report research report RR-78-4, International Institute of Applied Systems Analysis (1978)
9. Ruszczyński, A.: A regularized decomposition method for minimizing a sum of polyhedral functions. Math. Program. **35**(3), 309–333 (1986)
10. Hintermüller, M.: A proximal bundle method based on approximate subgradients. Comput. Optim. Appl. **20**(3), 245–266 (2001)
11. Kiwiel, K.C.: A proximal bundle method with approximate subgradient linearizations. SIAM J. Optim. **16**(4), 1007–1023 (2006)
12. Oliveira, W., Sagastizábal, C., Lemaréchal, C.: Convex proximal bundle methods in depth: a unified analysis for inexact oracles. Math. Program. **148**, 241–277 (2014)
13. Oliveira, W., Sagastizábal, C., Scheimberg, S.: Inexact bundle methods for two-stage stochastic programming. SIAM J. Optim. **21**(2), 517–544 (2011)
14. Higle, J.L., Sen, S.: Finite master programs in regularized stochastic decomposition. Math. Program. **67**(1–3), 143–168 (1994)
15. Lemaréchal, C., Nemirovskii, A., Nesterov, Y.: New variants of bundle methods. Math. Program. **69**(1), 111–147 (1995)
16. Kiwiel, K.C.: Proximal level bundle methods for convex nondifferentiable optimization, saddle-point problems and variational inequalities. Math. Program. **69**(1), 89–109 (1995)
17. Fábián, C.I.: Bundle-type methods for inexact data. CEJOR **8**(1), 35–55 (2000)
18. Oliveira, W., Sagastizábal, C.: Level bundle methods for oracles with on-demand accuracy. Optim. Methods Softw. **29**(6), 1180–1209 (2014)
19. Wolf, C., Fábián, C.I., Koberstein, A., Suhl, L.: Applying oracles of on-demand accuracy in two-stage stochastic programming-a computational study. Eur. J. Oper. Res. **239**(2), 437–448 (2014)
20. Fábián, C.I., Szőke, Z.: Solving two-stage stochastic programming problems with level decomposition. CMS **4**, 313–353 (2007)
21. Forsgren, A., Gill, P.E., Wong, E.: Primal and dual active-set methods for convex quadratic programming. Math. Program. **159**, 469–508 (2016)
22. Nocedal, J., Wright, S.J.: Numerical Optimization, 2nd edn. Springer, New York (2006)
23. Hintermüller, M., Ito, K., Kunisch, K.: The primal-dual active set strategy as a semismooth Newton method. SIAM J. Optim. **13**(3), 865–888 (2002)
24. Curtis, F.E., Han, Z., Robinson, D.P.: A globally convergent primal-dual active-set framework for large-scale convex quadratic optimization. Comput. Optim. Appl. **60**, 311–341 (2015)
25. Curtis, F.E., Han, Z.: Globally convergent primal-dual active-set methods with inexact subproblem solves. SIAM J. Optim. **26**(4), 2261–2283 (2016)
26. Dorn, W.S.: Duality in quadratic programming. Q. Appl. Math. **18**(2), 155–162 (1960)
27. Ito, K., Kunisch, K.: The primal-dual active set method for nonlinear optimal control problems with bilateral constraints. SIAM J. Control. Optim. **43**(1), 357–376 (2004)
28. Birge, J.R., Louveaux, F.: Introduction to Stochastic Programming. Springer, New York (2011)
29. Ruszczyński, A., Shapiro, A.: Stochastic programming models. Handb. Oper. Res. Manag. Sci. **10**, 1–64 (2003)
30. Wets, R.R.-J.: 3. In: Ermoliev, Y., Wets, R. (eds.) Large Scale Linear Programming Techniques in Stochastic Programming. Springer (1988)
31. Zakeri, G., Philpott, A.B., Ryan, D.M.: Inexact cuts in benders decomposition. SIAM J. Optim. **10**(3), 643–657 (2000)

32. Gangammanavar, H., Liu, Y., Sen, S.: Stochastic decomposition for two-stage stochastic linear programs with random cost coefficients. INFORMS J. Comput. **33**(1), 51–71 (2021)
33. Florescu, I., Tudor, C.A.: Handbook of Probability. Wiley (2013)
34. Higle, J.L., Sen, S.: Stochastic Decomposition: A Statistical Method for Large Scale Stochastic Linear Programming. Kluwer Academic Publishers, Boston (1996)
35. Christie, R.: Power Systems Test Case Archive, 1999 (2015)
36. Gassmann, H.I.: 2. The SMPS Format for Stochastic Linear Programs, pp. 9–19. https://doi.org/10.1137/1.9780898718799.ch2. https://epubs.siam.org/doi/abs/10.1137/1.9780898718799.ch2
37. Shapiro, A., Dentcheva, D., Ruszczynski, A.: Lectures on Stochastic Programming: Modeling and Theory, 3rd edn. Society for Industrial and Applied Mathematics, Philadelphia (2021). https://doi.org/10.1137/1.9781611976595

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.